MSML642 Homework 3: Autonomous Green-Gate Navigation with a TurtleBot3

Abigail Basener

10/14/2026

1 Part 1: Reference Demonstration

1.1 Environment and Setup

We set up the Docker environment on a Linux Mint 22.3 (Ubuntu 24.04 base) machine. We ran ROS 2 Humble, Gazebo, and the course's `msml642-hw3:humble` Docker image to build the full simulation and the needed interfaces. We loaded http://localhost:6080/vnc.html to view the Gazebo and RViz programs over VNC, where we could watch the simulation of the TurtleBot3 Waffle Pi in real time. The HW3 setup includes a built-in evaluator that tracks the simulation. It also starts the chosen policy to autonomously navigate the maze. The evaluator also stops the simulation when the robot gets to the goal or if a rule is broken, and it reports any violation and general information about the run, like distance and time. Each run can be on one of three maps, A, B, or C, which are explained further below.

We started with version 1 of the homework. However, the simulation had a real-time restriction, with a 420 s wall-clock limit. This was an issue since the machine we are using is a bit slower: it runs without a GPU, so all imaging was done on the CPU, taking up resources needed for the rest of the simulation. The machine was not able to execute all of the needed simulation time within the real-world limit. We then updated to the second version, which was limited by simulation time rather than real-world time, with a 300 s simulation-time limit. This allowed the machine to run the full amount of simulation time needed to complete a run.

We first verified that Part 1 ran correctly and that all of the setup had been done correctly. Then we broke Part 2 into two phases. The first phase focused on sensor interpretation. This gave us a map based on the fusion of the different sensors on the TurtleBot, as well as the odometry's knowledge of its position and past. Phase 2 was to take that map and, using rule-based logic, build the planning center of the robot. This has three modes based on where the robot is in the maze, and it uses the mode, the known map, and the unknown spaces around it to choose its behavior.

We edited only the `hw3_policy.py` file. All other files set up and run the simulation, and we did not edit any of those files, to keep the base simulation the same for all runs with the new policy.

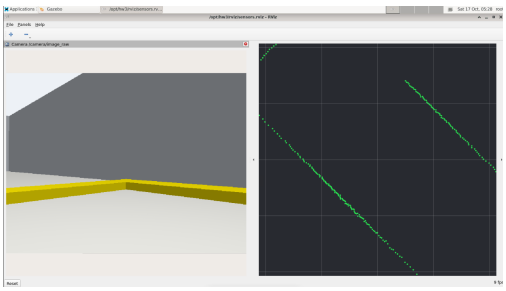
1.2 Interface Check

Table 1: Interface check before the Part 1 runs, and measured sensor rates during them. No stream went stale on either map.

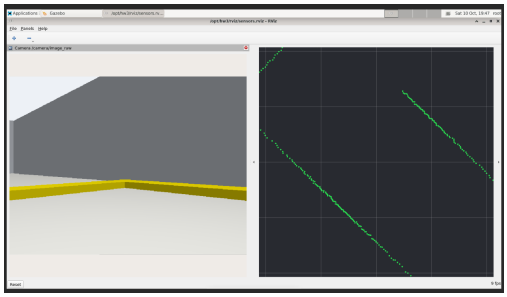
Topic	Frame	Format	maze_a (Hz)	maze_b (Hz)
`/odom`	`map`	child `base_footprint`, start $(0, -4.4)$	12.75	15.66
`/scan`	`base_scan`	360 samples, 0.12–3.5 m	4.34	5.32
`/camera/image_raw`	`camera_rgb_frame`	640×480, rgb8	3.17	4.34
`/cmd_vel`	—	1 bridge subscriber	—	—

Table 2: Reference demo results (classical policy)

Map	Success	Termination	Sim time (s)	Path (m)	Green	Red viol.
maze_a	Yes	success	147.64	20.12	Passed once	None
maze_b	Yes	success	68.16	10.68	Passed once	None



(a) maze_a



(b) maze_b

Figure 1: Part 1 final dashboard view, after the robot reaches the goal

1.3 Notes and Issues

We initially started with the first version of the homework, which, as mentioned, timed out. Once the evaluator was updated in version 2 to time out based on simulation time rather than real time, we were able to complete the mazes for Part 1. We also ran into an issue with stale sensor readings, caused by the simulation running slowly, which meant the camera did not produce an image at least once every 3 seconds of real time. We had to clean up some background processes, and then the machine had enough processing power and speed to run the simulation fast enough that the stale sensor reading was fixed.

2 Part 2: Policy Design

2.1 System Overview and Data Flow

Our robot has a camera facing forward, a LiDAR system on top with a 360-degree view up to 3.5 meters, and the robot itself knows where it is in the map frame. We can access these sensors with `/odom`, `/scan`, `/camera/image_raw`, and `/camera/camera_info`. This is the data that comes into

the robot, and these are our observations. To make sure all observations are fresh, we make sure all data points are at most 2 seconds old and all from after the last move was completed.

Then, using the sensor information, we need to map the environment around us. We can use the LiDAR to know where objects are, the camera to know the color of the objects in front of us, and the odometry to know where we are. Using these, we make the needed map updates.

Lastly, we have to plan and act on the new map and information. We use policies to make decisions about how we should act based on that knowledge in the planning stage. Then, we act on those plans, moving to the new location along the planned path with `/drive_to_pose` (Figure 2).

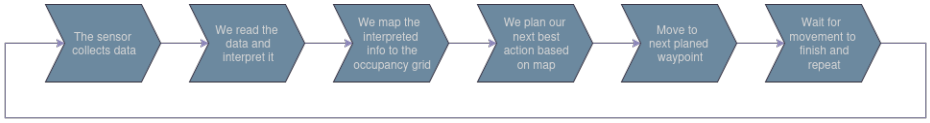


Figure 2: Steps in our robot environment mapping and planning

2.2 Sensor Processing

2.2.1 Odometry

This tracks the pose and yaw of the robot in the map frame. We know that we start at $(0, -4.4)$ and we know the goal is at $(0, 4.4)$, so keeping track of our location is important both for guiding us to the goal and for tracking past movement, for smarter decisions that keep the robot from getting stuck in a loop or dead end. We use Equation (1) to convert between the occupancy grid and the real location that the odometry reports, where $(x_0, y_0) = (-6, -6)$ is the corner of the grid and $r = 0.05\,\text{m}$ is the size of one cell.

$$\text{col} = \left\lfloor \frac{x - x_0}{r} \right\rfloor, \qquad \text{row} = \left\lfloor \frac{y - y_0}{r} \right\rfloor \tag{1}$$

2.2.2 LiDAR and the Occupancy Grid

The LiDAR gives a 360-degree view around the robot at a single z level. The beams start from the sensor going out, so the farther out, the more spread out the beams are. This will be important for understanding how reliable the readings are. The LiDAR can report back from a maximum of 3.5 meters.

We then used the LiDAR readings to fill out an occupancy grid. This grid is the size of the distance from the start to the goal plus a margin. The actual maze is smaller than this, but it makes sure that we have room for all space accessible to the robot to be represented in the grid. Each cell in the grid represents a $0.05\,\text{m}$ square in the real world. Each square can have one of five labels: unknown, which is what everything starts as; open, which is when there is evidence that a LiDAR beam passes through the space without hitting anything; or wall, which is when a beam hits something in that cell. Then there are gates, which are additional subclasses of walls with integrated camera color information.

The LiDAR sensor has a $(-0.064, 0)\,\text{m}$ offset from the odometry placement of the robot, so we first have to adjust the incoming data to align with where our robot is. Then, for each beam, it can report hitting something in that $3.5\,\text{m}$ range or not report anything. Using Equation (2), we find the cells the beam travels through and/or hits. Here (x, y) is the location of the robot and ψ is its rotation, (o_x, o_y) is the LiDAR offset, and (x_l, y_l) is the resulting position of the LiDAR. For each beam i, θ_i is its angle and d_i is the distance it was able to cover unobstructed, which ends

at (x_e, y_e). $\theta_{\min}$ is the angle of the first beam and $\Delta\theta$ is the angle between neighboring beams. Whether there was a hit or not, we know everything before that end point was open.

$$\begin{aligned} x_l &= x + o_x \cos\psi - o_y \sin\psi, \quad y_l = y + o_x \sin\psi + o_y \cos\psi \\ \theta_i &= \theta_{\min} + i\,\Delta\theta \\ x_e &= x_l + d_i \cos(\psi + \theta_i), \qquad y_e = y_l + d_i \sin(\psi + \theta_i) \end{aligned} \tag{2}$$

Each time a cell is reported as open, we add $+1$ to its score, and each time it is reported as having something in it, we add -3. Since hitting a wall is more dangerous than staying away from an open square, and a false wall reading is less likely than a false open reading, we weighted the wall result higher. Then any cell whose value is at or below -3 we call a wall, and any at or above $+2$ is an open area. As the beams spread out farther from the sensor, there are fewer beams going through any given cell, so closer cells will have a larger score change. This is a functional representation of the idea that the closer something is, the more confident the robot can be about it. These scores gave us a base map of the objects in the space in the occupancy grid.

2.2.3 Camera and Color Fusion

Our camera has an offset of $(0.076, 0)$ m, placed at the front of the robot, with a $\pm 31^\circ$ field of view. We use this information to correct for the offset of the sensor position so we can directly compare the data. We then took a ± 20 pixel band above and below the level that the LiDAR is at. This band contains the pixels that represent the walls that the LiDAR is hitting, and it removes the floor and sky so we can see just the color of the objects. We project the LiDAR beams onto this image so we can see what colors they are colliding with. We give each end of a beam projection an 8 pixel padding and require at least 50% of the area to be colored for it to pass as a colored object. We then pull a color mask from this, looking at what color the pixels are (red, green, or other), and use that to more persistently label a wall or colored gate.

The camera can see farther out than the LiDAR, so it often sees the gate before the LiDAR. In that case, it cannot update that wall as a gate and needs to wait for the robot to get closer, so both the LiDAR and camera observe the object. So we also have a layer of the occupancy grid for camera-seen that records where the camera has seen. This seen-by-camera layer is where we can mark any headings of colors that are important, such as red or green, that do not yet have a position since they are past the LiDAR. We did not implement this into the planner, but it is set up for further use and debugging.

Since the camera can be a bit slower to work and report than the LiDAR and odometry, we also have to check the timeliness of the outputs. We check that camera views are only fused with the LiDAR if they are within 0.1 seconds of the LiDAR reading. Due to slow simulation on the machine, they sometimes fall out of sync, and they would cause errors if unverified, if the robot moved between the two readings.

2.2.4 The Combined Map

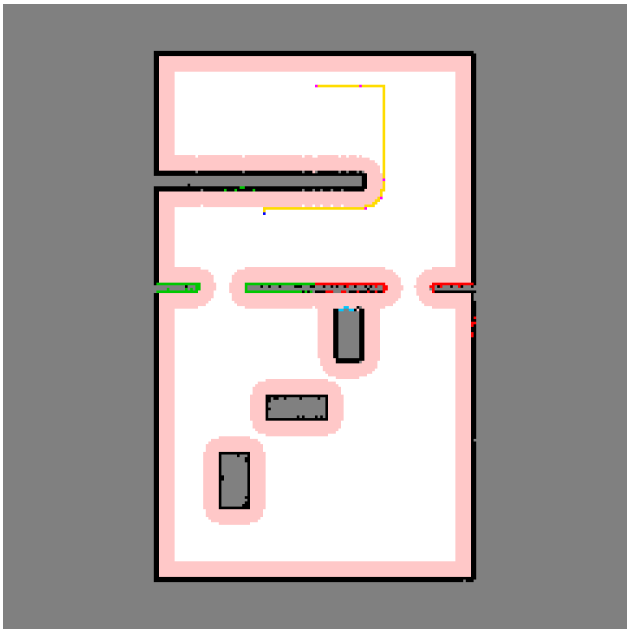


Figure 3: Camera and color fusion with padding of a fully explored maze A

Together, this gives us a map for maze A that looks like Figure 3. The gray is unknown, black is walls, white is open, green and red are where the robot thinks the gates are, and the pink is additional padding for clearance, so the robot does not collide with or get stuck on a wall. You can see in this map how there are some wall errors, where a wall is seen where it should be unknown, or open areas where there should be a wall. This gets better with more sensor sweeps, as the numbers update to the extremes of their values, drowning out a few bad readings or calculations where rounding or imperfect grid representations caused it to be registered in the wrong cell. We can also see some spillage of the green and red walls, where the camera saw a color but it was assigned to the wrong wall cell. Instead of spending time trying to clean up the sensor readings, our later planning logic accounts for these errors with a robust approach so it does not get confused. Sensor data will often have small inconsistencies, particularly when converting to a grid representation, and building a robust policy is smarter than trying to get all the errors out of the data.

2.3 Handling Move Results

After each move, one of three responses is sent back: OK, BLOCKED, or STALLED. These tell us how the robot is doing. OK means the robot reached the waypoint and everything worked as expected. BLOCKED means that the controller stopped because something was in the way, like a wall. STALLED is sent back when the robot stops making progress.

When we get an OK back from the previous move, we know everything is ready for the next cycle, and the robot waits for updated sensor readings so the process can start again. If a move comes back as BLOCKED or STALLED, we add that cell location to a blacklist. This makes it so that when we replan, we do not try to send the robot to the same cell that gave us problems again. If a BLOCKED or STALLED response is sent 4 times in a row, the policy stops, since there is a bigger issue with the logic or setup of the robot control.

2.4 State Representation

Table 3 shows all the representations of the environment, including the history that the sensors have collected, information about where the robot was and started, and the state the robot is currently functioning in.

Table 3: Persistent state

Array / field	Type	Meaning
Occupancy grid layers		
Scores	float32 grid	LiDAR scores per cell; starts at NaN if not seen
Labels	uint8 grid	Simple labels: unknown, free, wall, red gate, green gate
Flags	uint8 grid	Camera seen, green seen, red seen (one bit each)
Traveled	int32 grid	History of where the robot has traveled, counting up from 0
Planner state		
Stage	string	Which stage the robot is currently in: find gate, cross gate, to goal
Gate	Gate	The confirmed green gate
Start	(x, y)	Start position, used later to know which side of the gate is the far side
Gate lines	bool grid	Gate lines treated as a wall for red, and a one-way wall for green
Blacklist	list	Additional flagged cells for failed targets
Guide cell	cell	The current target

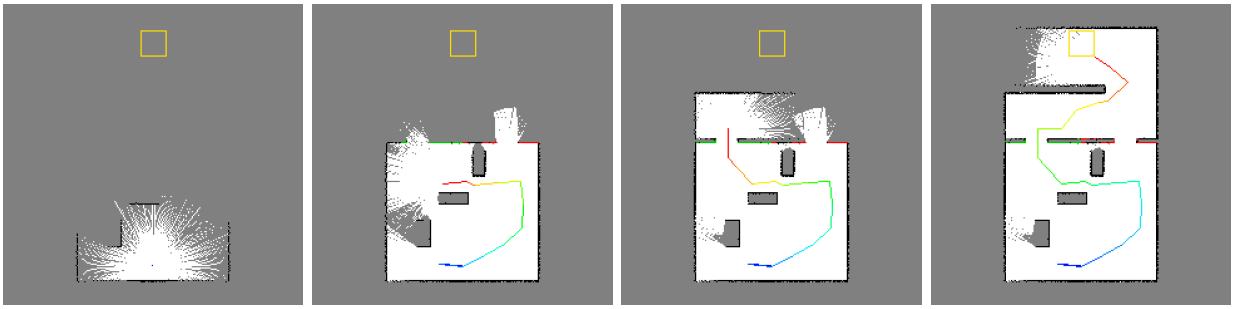
2.5 Policy

2.5.1 Stages

Table 4: Planning stages

Stage	Description
Find Gate	The robot does not know where the green gate is and needs to find it without going through the red gate.
Cross Gate	The green gate has been found. The robot needs to move to it and then align itself to go through the gate without hitting the wall.
Move to Goal	The green gate has been passed. Now the robot needs to explore, looking for the goal at $(0, 4.4)$, without crossing any more gates.

Figure 4 shows the three phases of the planning and the initial state the robot starts with. Like Figure 3, these are representations of the occupancy grid at the start of their phase. They also show a line representing where the robot has traveled, colored according to time in the simulation.



(a) First look (b) Gate confirmed (c) Gate crossed (d) Final

Figure 4: Map and layout at each stage

A human would be able to see the gate before the robot does. We are only using sensor fusion, which means we are limited by the LiDAR distance, so even though the robot sees the gate, it has to drive closer to the green until the LiDAR can see it as well. Then the next stage can start, where it lines up facing perpendicular to the gate and crosses it. Once it has gone through the gate, it moves toward the goal.

Each stage is a mini goal for the robot on the way to the final end goal. It starts just exploring. Then, when the green gate is discovered, it switches to the Cross Gate stage, where it sets up to align with the gate and crosses it. This switch happens when the green gate is detected in two rounds of planning in a row, which helps keep outliers or bad data from prematurely switching to the next stage. Once past the gate (by more than 0.2 meters past the opening), it begins the final stage, Move to Goal, where it continues to explore until it sees a known open path to the goal and drives directly to it.

2.5.2 Frontier Exploration and Scoring

This is how the robot moves when there is not a designated goal, such as a found gate or a direct path to the goal. In this mode, the robot looks at the occupancy grid for where the most unknown meets open, which means the robot can drive to that area. We want to learn the most about the maze in the least time and movement, so we score each frontier with Equation (3).

$$S(f) = w_s\,|f| \;-\; w_d\,\ell(f) \;-\; w_g\,\|\mathbf{c}_f - \mathbf{g}\| \;+\; \frac{w_{\text{green}}}{1+\delta_{\text{green}}(f)} \tag{3}$$

Here, $|f|$ is the number of cells on the front of the frontier. $\ell(f)$ is the distance to the target. This helps the robot stay on one target at a time so it does not waste time going back and forth, and it works to balance the amount explored with the time spent traveling. $\mathbf{c}_f$ and $\mathbf{g}$ are the locations of the target and the goal, respectively, to push the scoring toward desired outcomes. δ_{green} is another distance, to the nearest green cell, that helps push the exploration toward the green gate, in Find Gate mode only. Lastly, w_s, w_d, w_g, and w_{green} are all weight parameters that we can adjust to make the score's dependence on any one aspect more or less. Together, this score tells us the trade-off of going to any one cell and lets us choose the highest scoring cell to drive to next.

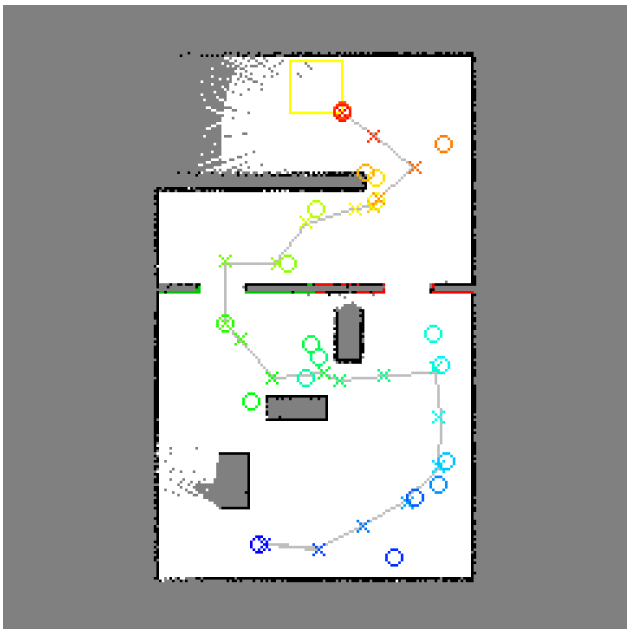


Figure 5: The occupancy grid at the end of the run, overlaid with the guide points (O) and the waypoints (X) throughout the robot's journey

In Figure 5, we can see how the robot paths around. In the first part of the maze in particular, we can see how the frontier mode works. First, the robot explores the closer area to the right, then it moves left, moving up to see new area. It sees the red gate first and avoids it, so it moves to the left and sees the green. Once the green is seen but the gate is not "discovered" yet, it explores with a bias pulling it toward the green pixels. Once it has explored more near the gate, it finds it and starts the next stage.

2.5.3 Gate Detection

In this stage, the robot first checks that it has the real gate and not a fragmented sensor reading. It does this by making clusters of green gate wall cells. It looks for groups of at least 4 cells, which are large enough to be more than a bad reading. We then look for two groups separated by open space the robot can drive through.

$$(\mathbf{a}^\star, \mathbf{b}^\star) = \arg\min_{\mathbf{a}\in A,\, \mathbf{b}\in B} \|\mathbf{a}-\mathbf{b}\|, \qquad w = r\,\|\mathbf{a}^\star - \mathbf{b}^\star\| \tag{4}$$

$$\varphi = \frac{\#\{\text{FREE cells on the line } \mathbf{a}^\star \to \mathbf{b}^\star\}}{\#\{\text{cells on that line}\}} \tag{5}$$

$$\mathbf{c} = \tfrac{1}{2}(\mathbf{a}^\star + \mathbf{b}^\star), \qquad \mathbf{u} = \frac{\mathbf{b}^\star - \mathbf{a}^\star}{\|\mathbf{b}^\star - \mathbf{a}^\star\|}, \qquad \mathbf{n} = (-u_y,\ u_x) \tag{6}$$

Here, A and B are two possible color clusters, after merging any fragmented colored walls within 2 cells of each other to help smooth out some missing color detections. $\mathbf{a}^\star$ and $\mathbf{b}^\star$ are the closest cells of the two clusters, which will be the inner sides of the gate to pass through. w is the width of the gap in the gate, to check that the robot will fit. $\mathbf{c}$ is the center of that gap, $\mathbf{u}$ is the unit vector crossing the gap, and $\mathbf{n}$ is the normal vector to that, which the robot will drive on. φ is the fraction of cells on the gate's gap line that are seen as open by the LiDAR. The gate is seen as

valid if $0.45 \leq w \leq 1.6\,\text{m}$ and $\varphi \geq 0.6$. All of these steps and tolerances protect against false gate detection by making sure the gap and cluster sizes are reasonable.

2.5.4 Gate Crossing

Once the gate has been found, the robot needs to cross it. It draws a line between the two green groups that represent the two sides, $\mathbf{u}$. It then draws a perpendicular line, called $\mathbf{n}$, that starts at $\mathbf{p}_a$ and ends at $\mathbf{p}_p$. Then the robot drives to $\mathbf{p}_a$, turns to face the gate, and drives to $\mathbf{p}_p$. At this point, the robot has crossed the green gate and starts the next phase, and the green gate is now treated like a wall to prevent going back through it. D_a and D_p are the distances from the gate that the robot needs to start and end at, to be safely away from it when turning so it does not get stuck. σ tracks which side of the gate is the start side, which should have $\mathbf{p}_a$ placed on it.

$$\mathbf{p}_a = \mathbf{c} + \sigma D_a \mathbf{n}, \qquad \mathbf{p}_p = \mathbf{c} - \sigma D_p \mathbf{n}, \qquad D_a = D_p = 0.6 \text{ m} \tag{7}$$

2.5.5 Reaching the Goal

If the goal is reachable through the known part of the map, the robot drives to the closest known cell of the goal. If the goal is not reachable yet, the robot goes back into frontier mode while also being pulled toward the goal using the scoring in Equation (3). As it does this, it sees more of the maze and continues to find new cells to move to until it gets to the goal. This may lead the robot into a dead end, so there are further policies for those cases that keep it from getting stuck in a path that is leading toward the goal but closed off.

2.6 Path Planning

First, we have to know where the center of the robot is compared to the odom sensor. We found the center is 0.064 m behind the odom sensor, which is important for knowing where the body of the robot is. When the robot turns, it turns around the odom point, not its center of mass. The farthest point of the robot from the odom point is a back corner, which is $0.064 + 0.1325\,\text{m}$ behind it (the center offset plus half the 0.265 m base) and 0.153 m to the side (half the 0.306 m width, including the wheels). Using Equation (8), we find that this corner is $\approx 0.25\,\text{m}$ from the odom point. We know that if we stay at least that far away from the walls in every direction, we can safely turn to any angle. We then add a safety margin of $m = 0.05\,\text{m}$, and this gives us a final clearance of $R = 0.30\,\text{m}$, which is 6 cells in our grid.

$$r_{\text{robot}} = \sqrt{(0.064 + 0.1325)^2 + 0.153^2} \approx 0.25 \text{ m}, \qquad R = r_{\text{robot}} + m = 0.25 + 0.05 = 0.30 \text{ m} \tag{8}$$

We then use this knowledge to pick the highest scoring cell in frontier mode, or for our other modes of crossing the gate or moving directly to the goal, to choose cells that are at least 6 cells away from a wall.

2.7 Replanning

The robot replans at each move, restarting the cycle. It waits for new data and updates the occupancy grid with the new information. It may also add cells to the blacklist if they were unreachable in the last movement attempt, and the robot checks if it needs to update its state if the criteria are met.

2.8 Safety and Recovery

When the robot gets stuck, whether it is blocked by a wall or cannot reach a planned cell, we have a few ways to handle this. As mentioned above, if the movement comes back as BLOCKED or STALLED, the robot blacklists the cell that caused the issue and replans so that a new cell is selected. If the robot moves too close to a wall, it then moves to the nearest safe cell and replans. If there are no cells to move to, it turns and looks around. These all help the robot find new and safe cells to travel to, which helps it get to the goal when it comes across trickier, unforeseen issues.

Recovery case study. We had an issue where the robot kept sending new moves to the controller but was not moving. We let it run, and it repeated about 200 times before we stopped it.

Listing 1: Recovery case log excerpt

```
[policy_example]: ODOM x=0.40 y=-1.25 yaw=133.3deg frame=map
[policy_example]: RECOVERY: escaping to safe cell (0.38, -1.22)
[policy_example]: Waypoint: status = 4, code=OK
[policy_example]: RECOVERY: escaping to safe cell (0.38, -1.22)
[policy_example]: Waypoint: status = 4, code=OK
[policy_example]: RECOVERY: escaping to safe cell (0.38, -1.22)
```

We found this was because the robot had stopped one cell inside the padded area around a wall. The closest safe cell was 3.6 cm away, but the controller has a tolerance of about 8 cm. So the plan was to move to the safe cell, but the controller considered the robot already there. This caused an infinite loop, with the controller replying OK without moving and the planner still wanting to move to the safe cell.

To fix this, if the safe cell is within 0.12 m, which is higher than the tolerance of the controller, we treat the robot as if it were already at that safe cell and plan from there. This way, the planner only tells the robot to make escape moves that are larger than the minimum movement the controller will act on. This fixed the case, and if it were to happen again, the robot has a policy to deal with it. We reran the simulation and never hit this issue again in our 45 tests after the fix.

3 Two-Value Parameter Comparison

3.1 Setup

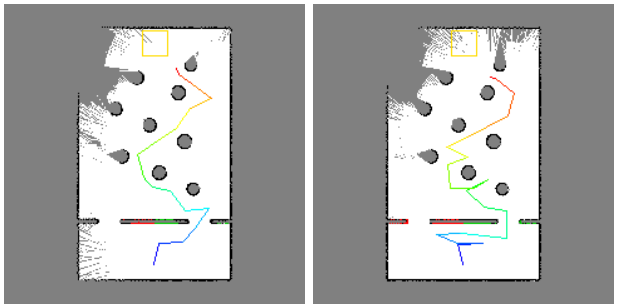
We decided to update the w_g value. This is the parameter that pulls the frontier mode toward the goal in Equation (3). We started with it at 0.3, and we wanted to see what would happen if we increased this value. By increasing it, the robot should go more directly to the goal. So we increased the value to 1.0, which should cause a more direct path to the goal.

3.2 Results

In Table 5, we can see a direct comparison of these values, with 0.3 being the original and 1.0 the alternative. We can see they both finish maze_b, with the alternative $w_g = 1.0$ moving slightly more directly, causing a faster run. We can see how the robot acted differently in Figure 6, where the 1.0 has a much more direct path with less wandering back and forth.

Table 5: Goal weight w_g: 0.3 (original) vs. 1.0 (alternative) on maze_b, one run each

Metric	$w_g = 0.3$	$w_g = 1.0$
Success	Yes	Yes
Sim time (s)	99.99	83.41
Path length (m)	19.18	14.87
Collisions	0	0



(a) $w_g = 1.0$ (b) $w_g = 0.3$

Figure 6: Final maps on maze_b for the two values of w_g

3.3 Discussion and Choice

We chose to stick with $w_g = 0.3$. Even though $w_g = 1.0$ worked better for maze_b, it can have issues of getting stuck more easily, since it takes less time exploring alternative routes. Since we have three mazes, we wanted to keep w_g more generic, for a general maze that may have dead ends or other more complicated layouts.

This would be a place where it helps to know the robot's full goal, past simply finding the goal. If it were working in real-world places like personal homes, there may be more places like closets to get stuck in, so it would need more exploration. A robot working in a school with long, straight, connected corridors and closed doors might work better with a higher value, since there is less of a chance to get stuck and thus less of a need to explore every corner.

4 Part 2 Results

4.1 Submitted Runs

Table 6: Evaluator results for the submitted (recorded) runs of the final policy. maze_c is the bonus map.

Map	Success	Sim time (s)	Path (m)	Green	Red viol.	Rev. green	Collision
maze_a	Yes	81.00	17.92	Crossed	None	None	None
maze_b	Yes	86.07	15.71	Crossed	None	None	None
maze_c	Yes	110.53	17.67	Crossed	None	None	None

4.2 Evaluator Metrics

Table 7: Batch results: 15 runs per map from a fresh start. $\bar{t}$ = mean simulation time, $\bar{L}$ = mean path length (both over successful runs, mean ± std).

Map	Success	$\bar{t}$ (s)	$\bar{L}$ (m)	Green	Red viol.	Rev. green	Collision	Timeout	Goal
maze_a	15 / 15	86.8 ± 15.6	18.1 ± 3.6	All	None	None	None	None	All
maze_b	13 / 15	99.1 ± 10.9	18.3 ± 1.9	14	1	None	None	1	13
maze_c	15 / 15	108.9 ± 25.0	18.3 ± 3.4	All	None	None	None	None	All

4.3 Reliability (Batch Runs)

We can see in Table 7 that the policy is consistent, failing 2 times out of the 45 runs on 3 different maze layouts. One of these is a timeout issue, the robot taking too long, and one is due to a red violation. The red violation was caused by the robot's first frontier cell to move to being through the red gate, before the robot had time to detect that the red gate was there. With some better computer vision techniques, we may have been able to tell the robot where the gate is faster and prevent this issue. The timeout was because our robot spent too much time wandering around. Again, some better computer vision techniques may help the robot identify the gates and goals faster, giving it a better understanding of the map so it can move where it needs to go quickly and directly without as much wandering. Overall, the policy had a 95% success rate.

5 Limitations

Our biggest limitation is not having a more complex computer vision program. There are times, like on maze_b, where the red gate is close to the start. Since it takes a few sensor passes for the robot to detect a gate, needing both a camera and a LiDAR view, it is possible for the robot to randomly cross the red gate before it can detect it and set it as off-limits. It would be possible to use better computer vision to see, identify, and label the gate before our current version is able to, protecting against this case. This may also help find the green gate faster. Since we often can see it before the LiDAR can, better identification of the gates without the LiDAR would allow us to make faster and more direct decisions.

There are some assumptions about the maze. We have a set size for our occupancy grid that works for our three mazes, but future maps of different sizes may break this assumption. Having a grid that can grow with information may be needed for a more unknown space. However, such grids can be costly to grow. We also assume in our code that the start and the goal are known. If we were to use a new map with a different goal location, then our robot would not be able to finish consistently.

6 Recordings

We recorded running our new policy on maze_a and maze_b. These are the same runs that we submitted the screenshots, JSON, and text files for. These screen recordings are located on the linked GitHub, you have to download them to view.

- maze_a: https://github.com/abasener/MSML642-Homework-3/blob/main/Results/Part2/my_policy_maze_a.webm

- maze_b: https://github.com/abasener/MSML642-Homework-3/blob/main/Results/Part2/my_policy_maze_b.webm

A small note: these runs are a bit slower, which seems to be because of the additional screen recording. The other tests, for the parameter comparison and the 15 runs on each maze, ran with higher sensor rates.

7 Bonus: Maze C (Optional)

7.1 Maze C Completion

We ran the policy on maze C. In Table 7, we can see that the robot reliably drove successfully through the maze 15 out of 15 times tested. Our policy worked well on this maze, handling it a bit better than it handled maze_b.

8 JSON and Files

As well as being submitted in the zip folder on ELMS, all materials can be found at https://github.com/abasener/MSML642-Homework-3/tree/main.

9 Parameter Values

Table 8: Tunable parameters and final values

Name	Value	Name	Value	Name	Value
Robot and motion					
ROBOT_RADIUS	0.25 m	SAFETY_MARGIN	0.05 m	MAX_SENSOR_AGE_S	2.0 s
MAX_STEP	0.5 m	MIN_STEP	0.10 m	MAX_LEG	1.0 m
MAX_FAILS	4	ESCAPE_MIN	0.12 m	GOAL_XY	(0, 4.4)
Occupancy grid (LiDAR)					
res	0.05 m	Xmin, Xmax	±6.0 m	Ymin, Ymax	±6.0 m
LIDARosX, Y	(−0.064, 0) m	InfFreeRang	3.0 m	scoreMin, Max	±10
hitEv	−3	freeEv	+1	wallThr, freeThr	−3, +2
Camera and color fusion					
camOSx, y	(0.076, 0) m	horBand	20 px	colorFracThr	0.5
edgeGuard	8 px	maxCamLag	0.1 s	greenMargR, B	30, 20
redMargG, B	30, 20	yelMargB, MaxRG	60, 50 (debug)		
Frontier scoring					
W_SIZE	0.1	W_DIST	1.0	W_GOAL	0.3
W_GREEN	4.0	HYST	3.0	KEEP_R	0.3 m
RL_RADIUS	0.5 m	MIN_FRONTIER	5 cells		
Gate detection and crossing					
GATE_MERGE	2 cells	GATE_MIN_EDGE	4 cells	GATE_CONFIRM	2
GATE_MIN_GAP	0.45 m	GATE_MAX_GAP	1.6 m	GATE_FREE_FRAC	0.6
APPROACH_D	0.6 m	PASS_D	0.6 m	APPROACH_TOL	0.2 m
ALIGN_TOL	0.35 rad				